



Course	Database Management Systems
Phase	Final Project
Submitted by	ADIL ALI & SANAULLAH
SAP ID	70174929 70172228
Submitted to:	Ms: FATIMA JAVED

E-Ticket Booking System

1. Introduction

The E-Ticket Booking System is a digital platform designed to make transport management simple and efficient. In traditional setups, managing passenger records, bus schedules, routes, and ticket availability manually can lead to human error, double-bookings, and slow service.

This project solves those problems by creating a centralized database that connects passenger data, vehicle schedules, and real-time seat tracking. The goal is to ensure that a passenger can quickly find a route, check available seats, and book a ticket without any data confusion.

2. Key Objectives

The key objectives this project is covering are as follows:

- **Centralized Records:** To store all passenger, vehicle, and route details securely in one place.
- **Live Seat Management:** To track exactly how many seats are left on any given bus trip.
- **Clean Organization:** To structure the data logically so that information can be retrieved instantly without lag or repetition.

3. Entity Identification & Data Dictionary

Core Entities And Their Attributes

Entity	Attributes	Primary Key	Foreign Key	Description
Users	First_Name, Last_Name, City, User_Role, Phone_number	User_ID	NA	Captures generalized data for all system actors.
Vehicles	License_Plate, Vehicle_Type, Total_Capacity	Vehicle_ID	NA	Represents physical transit assets and maximum limits.
Routes	Origin_Station, Destination_Station , Estimated_Duration_Hours	Route_ID	NA	Defines the spatial pathways and endpoints.
Schedules	Vehicle_ID, Route_ID, Departure_Time, Ticket_Price, Available_Seats	Schedule_ID	Vehicle_ID, Route_ID	A temporal instance mapping a vehicle to a route.
Bookings	User_ID, Schedule_ID, Seats_Booked, Total_Amount, Booking_Status	Booking_ID	User_ID, Schedule_ID	The transactional record of a user claiming seats.
User_phone	User_ID, Phone_Number	UserID , Phone_number	UserID	A weak entity tracking bags tied to a booking.
Luggage	Booking_ID, Tag_Number, Waytek	Booking_ID, Tag_Number	Booking_ID	A weak entity tracking bags tied to a booking.

4. Relationships & Cardinalities

1. Vehicles to Schedules

- **Relationship:** Assigned_To
- **Rule:** One Vehicle can be assigned to multiple Schedules (at different times). A Schedule relies on exactly one physical Vehicle.
- **Cardinality:** One to Many (1:N)

2. Routes to Schedules

- **Relationship:** Defines
- **Rule:** A spatial Route can have multiple Schedules (e.g., Morning, Evening). A Schedule operates on exactly one Route.
- **Cardinality:** One to Many (1:N)

3. Users to Bookings

- **Relationship:** Places
- **Rule:** One User can register and place multiple passenger Bookings over time. However, a single transaction Booking is linked back to exactly one unique registered User.
- **Cardinality:** One to Many (1:N)

4. Schedules to Bookings

- **Relationship:** Contains
- **Rule:** One Schedule can receive multiple passenger Bookings (up to its maximum capacity). A single Booking is tied to exactly one Schedule.
- **Cardinality:** One to Many (1:N)

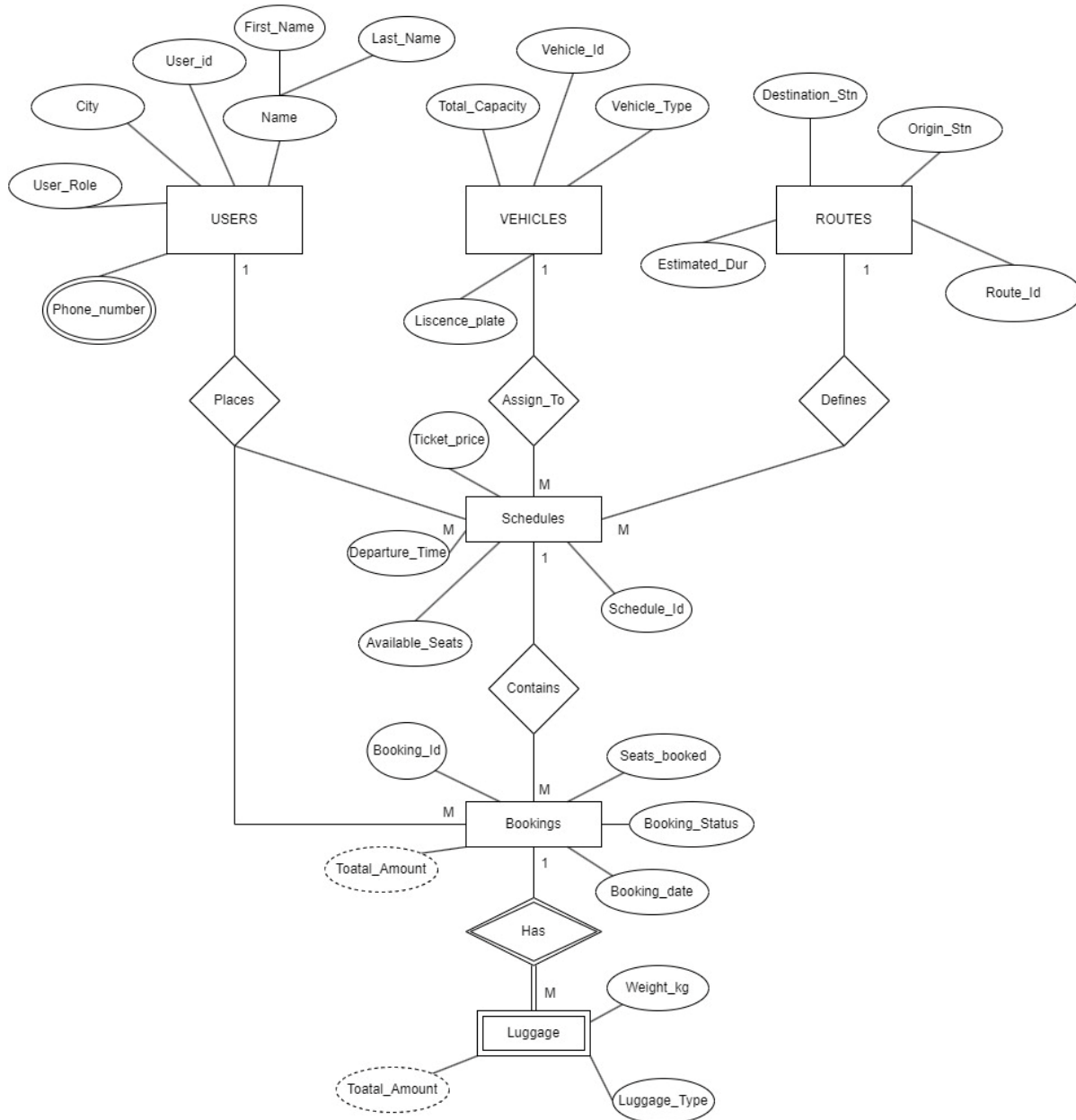
5. Bookings to Luggage

- **Relationship:** Has
- **Rule:** One Booking can have multiple pieces of luggage tied to it. A piece of luggage is a weak entity and belongs entirely to one parent Booking (enforced via ON DELETE CASCADE).
- **Cardinality:** One to Many (1:N)

6. Users to User_Phones

- **Relationship:** Possesses (*Derived from the multi-valued Phone_Number attribute*)
- **Rule:** One User can possess multiple active Phone Numbers in the system. A specific Phone Number entry belongs strictly to one designated User.
- **Cardinality:** One to Many (1:N)

5. Entity-Relationship Diagram



6.Mapped Tables of ER Diagram

User Table:

Column Name	Data Type	Key Type	Constraint / References	Description
User_ID	INT	PK	AUTO_INCREMENT	Unique identifier for each user
First_Name	VARCHAR(50)		NOT NULL	Part of the composite Name attribute
Last_Name	VARCHAR(50)		NOT NULL	Part of the composite Name attribute
City	VARCHAR(50)			Operational city location
User_Role	VARCHAR(20)		DEFAULT 'Passenger'	User category (e.g., Passenger, Admin)

User_Phone Table:

Column Name	Data Type	Key Type	Constraint / References	Description
User_ID	INT	PK, FK	References Users(User_ID)	Links phone number to a specific user
Phone_Number	VARCHAR(20)	PK	NOT NULL	Part of the composite primary key

Vehicles Table:

Column Name	Data Type	Key Type	Constraint / References	Description
-------------	-----------	----------	-------------------------	-------------

Vehicle_ID	INT	PK	AUTO_INCREMENT	Unique transport unit identifier
License_Plate	VARCHAR(20)		UNIQUE, NOT NULL	Vehicle physical tracking plate number
Vehicle_Type	VARCHAR(30)		NOT NULL	Category (e.g., Bus, Train, Flight)
Total_Capacity	INT		CHECK (Total_Capacity > 0)	Maximum available seats on board

Routes Table:

Column Name	Data Type	Key Type	Constraint / References	Description
Route_ID	INT	PK	AUTO_INCREMENT	Unique identifier for fixed paths
Origin_Station	VARCHAR(100)		NOT NULL	Trip departure location
Destination_Station	VARCHAR(100)		NOT NULL	Trip arrival endpoint location
Estimated_Duration	INT		Minutes	Expected travel time

Schedules Table:

Column Name	Data Type	Key Type	Constraint / References	Description
Schedule_ID	INT	PK	AUTO_INCREMENT	Specific timestamped trip entity
Departure_Time	DATETIME		NOT NULL	Planned trip leave date/time
Ticket_Price	DECIMAL(10,2)		CHECK (Ticket_Price >= 0)	Base currency cost for one seat
Available_Seats	INT			Dynamic countdown of unbooked spots
<i>Vehicle_ID</i>	INT	FK	References Vehicles(Vehicle_ID)	Pulled from the 1-side of

				<i>Assigned_To</i>
<i>Route_ID</i>	INT	FK	References Routes(Route_ID)	Pulled from the 1-side of <i>Defines</i>

Bookings Table:

Column Name	Data Type	Key Type	Constraint / References	Description
Booking_ID	INT	PK	AUTO_INCREMENT	Unique transaction receipt ID
Seats_Booked	INT		CHECK (Seats_Booked > 0)	Number of tickets bought together
Booking_Status	VARCHAR(20)		DEFAULT 'Pending'	(e.g., Confirmed, Cancelled, Paid)
Booking_Date	DATETIME		NOT NULL	Timestamp when booking was made
Total_Amount	DECIMAL(10,2)		Derived field	Stored for financial audit history
<i>User_ID</i>	INT	FK	References Users(User_ID)	Pulled from the 1-side of <i>Places</i>
<i>Schedule_ID</i>	INT	FK	References Schedules(Schedule_ID)	Pulled from the 1-side of <i>Contains</i>

Luggage Table:

Column Name	Data Type	Key Type	Constraint / References	Description
Booking_ID	INT	PK, FK	References Bookings(Booking_ID)	From parent entity; cascades on delete
Tag_Number	VARCHAR(50)	PK	Discriminator partial key	Barcode or unique physical label ID
Weight_KG	DECIMAL(5,2)		NOT NULL	Checked baggage bag weight mass
Luggage_Type	VARCHAR(30)			(e.g., Carry-on,

				Oversized, Standard)
--	--	--	--	-------------------------

7. Database Normalization

Normalization is the process of organizing data in a database to avoid duplication and protect data integrity. For this E-Ticket Booking System, the database was designed step-by-step to meet the Three Normal Forms (1NF, 2NF, and 3NF).

First Normal Form (1NF)

A database is in 1NF if all fields contain atomic (single) values and there are no repeating groups.

- **Before 1NF:** If we kept all data in one big spreadsheet, a single row might have a cell containing multiple phone numbers or multiple passenger names booked for the same trip.
- **After 1NF:** We ensured that every column in every table stores only one piece of data per row. For instance, each passenger in the Users table gets their own unique row with a single phone number and user ID.

Before 1NF:

User Table:

User_ID	First_name	Last_name	City	Phone_numbers	Role
1	Ali	Hamza	Lahore	0300-1112223, 042-9999999	passenger
2	Ayesha	Tariq	Lahore		passenger

After 1NF:

User Table:

User_ID	First_name	Last_name	City	Role
1	Ali	Hamza	Lahore	passenger
2	Ayesha	Tariq	Lahore	passenger

User_Phone Table:

User_ID(PK,FK)	Phone_numbers(PK)
1	0300-1112223
1	042-9999999
2	0321-4445556
2	0333-2222222

Second Normal Form (2NF)

A database is in 2NF if it is already in 1NF and all non-key columns depend entirely on the primary key.

- **The Problem:** If we put route details (like departure city and arrival city) inside the Bookings table, we would be constantly repeating the route names for every single passenger who books that trip.
- **The Solution:** We separated the data into independent tables based on their core purpose. Route details went into the Routes table, trip timings went into Schedules, and passenger info went into Users. They are linked together cleanly using Foreign Keys.

Before 2NF:

Bookings Table:

Booking_ID (PK)	User_ID	Seats_Booked	Total_Amount	Origin_Station	Destination_Station	Departure_Time
1	1	2	5000.00	Lahore Campus	Islamabad Campus	2026-06-10 08:00:00
3	1	3	3600.00	Lahore Campus	Faisalabad Terminal	2026-06-12 09:00:00

After 2NF:

Bookings Table:

Booking_ID (PK)	User_ID (FK)	Schedule_ID (FK)	Seats_Booked	Total_Amount
1	1	1	2	5000.00
3	1	3	3	3600.00

Schedules Table:

Schedule_ID (PK)	Vehicle_ID (FK)	Route_ID (FK)	Departure_Time	Ticket_Price
1	1	1	2026-06-10 08:00:00	2500.00
3	2	3	2026-06-12 09:00:00	1200.00

Routes Table:

Route_ID	Origin_Station	Destination_Station	Estimated_Duration_Hours
----------	----------------	---------------------	--------------------------

(PK)			
1	Lahore Campus	Islamabad Campus	4.50
3	Lahore Campus	Faisalabad Terminal	2.50

Third Normal Form (3NF)

A database is in 3NF if it is in 2NF and there are no transitive dependencies (meaning non-key columns do not depend on other non-key columns).

- **The Problem:** If the Bookings table included a column for Vehicle_Type or Total_Capacity, that data would actually depend on the vehicle itself, not on the booking transaction.
- **The Solution:** All vehicle attributes are confined strictly to the Vehicles table. The Bookings table only tracks transaction data (like the booking date and seat status) and uses IDs to point to the other tables. This satisfies 3NF perfectly.

Before 3NF:

Schedules Table:

Schedule_ID (PK)	Vehicle_ID	Departure_Time	Ticket_Price	Vehicle_Type	Total_Capacity
1	1	2026-06-10 08:00:00	2500.00	AC Coaster	25
2	5	2026-06-11 10:00:00	1800.00	Luxury Bus	40

After 3NF:

Schedules Table:

Schedule_ID (PK)	Vehicle_ID (FK)	Route_ID (FK)	Departure_Time	Ticket_Price	Available_Seats
1	1	1	2026-06-10 08:00:00	2500.00	25
2	5	2	2026-06-11 10:00:00	1800.00	40

Vehicle Table:

Vehicle_ID	License_Plate	Vehicle_Type	Total_Capacity
------------	---------------	--------------	----------------

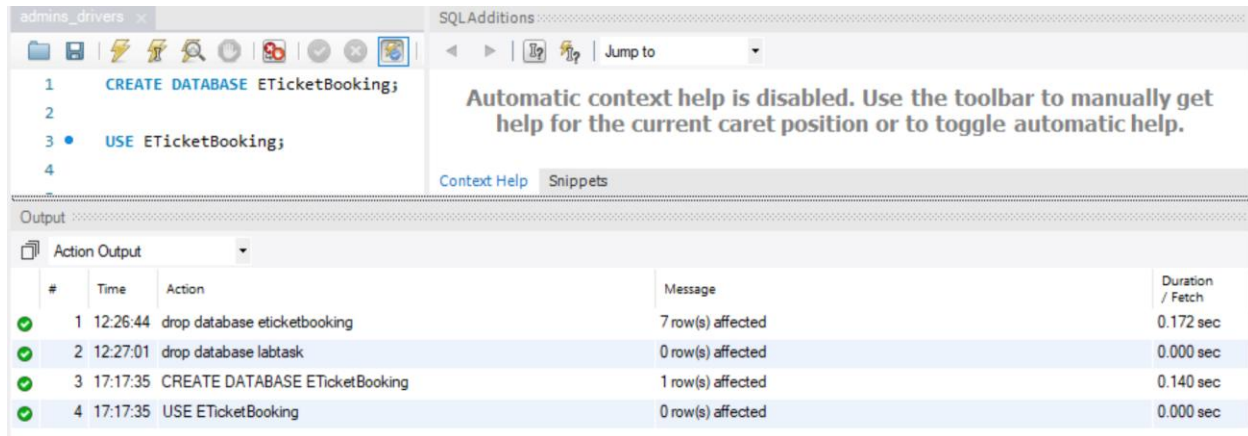
(PK)			
1	LHR-1001	AC Coaster	25
5	LHR-2005	Luxury Bus	40

8. Database Implementation (DDL & DML)

CREATE Tables

The following queries establish the relational foundation of the database, enforcing strict data typing, AUTO_INCREMENT sequence generation, and referential integrity through Foreign Keys.

```
CREATE DATABASE ETicketBooking;
USE ETicketBooking;
```



Create User Table

```
CREATE TABLE Users (
    User_ID INT AUTO_INCREMENT PRIMARY KEY,
    First_Name VARCHAR(50) NOT NULL,
    Last_Name VARCHAR(50) NOT NULL,
    City VARCHAR(50) NOT NULL,
    User_Role VARCHAR(20) DEFAULT 'Passenger'
);
```

Create Vehicles Table

```
CREATE TABLE Vehicles (
    Vehicle_ID INT AUTO_INCREMENT PRIMARY KEY,
    License_Plate VARCHAR(20) UNIQUE NOT NULL,
    Vehicle_Type VARCHAR(50) NOT NULL,
    Total_Capacity INT NOT NULL CHECK (Total_Capacity > 0)
);
```

Create Routes Table

```
CREATE TABLE Routes (
```

```
Route_ID INT AUTO_INCREMENT PRIMARY KEY,  
Origin_Station VARCHAR(100) NOT NULL,  
Destination_Station VARCHAR(100) NOT NULL,  
Estimated_Duration_Hours DECIMAL(4,2)  
);
```

Create Schedules Table

```
CREATE TABLE Schedules (  
    Schedule_ID INT AUTO_INCREMENT PRIMARY KEY,  
    Vehicle_ID INT NOT NULL,  
    Route_ID INT NOT NULL,  
    Departure_Time DATETIME NOT NULL,  
    Ticket_Price DECIMAL(10,2) NOT NULL CHECK (Ticket_Price > 0),  
    Available_Seats INT NOT NULL CHECK (Available_Seats >= 0),  
    FOREIGN KEY (Vehicle_ID) REFERENCES Vehicles(Vehicle_ID),  
    FOREIGN KEY (Route_ID) REFERENCES Routes(Route_ID)  
);
```

Create Bookings Table

```
CREATE TABLE Bookings (  
    Booking_ID INT AUTO_INCREMENT PRIMARY KEY,  
    User_ID INT NOT NULL,  
    Schedule_ID INT NOT NULL,  
    Seats_Booked INT NOT NULL CHECK (Seats_Booked > 0),  
    Total_Amount DECIMAL(10,2) NOT NULL,  
    Booking_Status VARCHAR(20) DEFAULT 'Confirmed',  
    Booking_Date DATETIME DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (User_ID) REFERENCES Users(User_ID),  
    FOREIGN KEY (Schedule_ID) REFERENCES Schedules(Schedule_ID)  
);
```

Create User_Phones Table

```
CREATE TABLE User_Phones (  
    User_ID INT NOT NULL,  
    Phone_Number VARCHAR(20) NOT NULL,  
    PRIMARY KEY (User_ID, Phone_Number),  
    FOREIGN KEY (User_ID) REFERENCES Users(User_ID) ON DELETE CASCADE  
);
```

Create Luggage Table

```
CREATE TABLE Luggage (  
    Luggage_ID INT AUTO_INCREMENT PRIMARY KEY,  
    Booking_ID INT NOT NULL,  
    Tag_Number INT NOT NULL,  
    Weight_KG DECIMAL(5,2) NOT NULL CHECK (Weight_KG > 0),  
    FOREIGN KEY (Booking_ID) REFERENCES Bookings(Booking_ID) ON DELETE CASCADE  
);
```


The screenshot displays a SQL IDE window titled 'admins_drivers'. The main editor shows a SQL script with the following content:

```

134
135     Tag_Number INT NOT NULL,
136
137     Weight_KG DECIMAL(5,2) NOT NULL CHECK (Weight_KG > 0),
138
139     FOREIGN KEY (Booking_ID) REFERENCES Bookings(Booking_ID) ON DELETE CASCADE
140
141 );
142
143 • CREATE TABLE Luggage (
144
145     Luggage_ID INT AUTO_INCREMENT PRIMARY KEY,
146
147     Booking_ID INT NOT NULL,
148

```

Below the editor is the 'Output' pane, which shows the 'Action Output' log. The log contains 12 entries, each with a status icon, a number, a time, an action, and a message. The final entry (12) shows an error: 'Error Code: 1050. Table 'luggage' already exists'.

#	Time	Action	Message
✓ 1	12:26:44	drop database eticketbooking	7 row(s) affected
✓ 2	12:27:01	drop database labtask	0 row(s) affected
✓ 3	17:17:35	CREATE DATABASE ETicketBooking	1 row(s) affected
✓ 4	17:17:35	USE ETicketBooking	0 row(s) affected
✓ 5	17:42:47	CREATE TABLE Users (User_ID INT AUTO_INCREMENT PRIMARY KEY, First_Name VARCHAR(50) NOT NULL, La...	0 row(s) affected
✓ 6	17:42:47	CREATE TABLE Vehicles (Vehicle_ID INT AUTO_INCREMENT PRIMARY KEY, License_Plate VARCHAR(20) UNIQUE NO...	0 row(s) affected
✓ 7	17:42:47	CREATE TABLE Routes (Route_ID INT AUTO_INCREMENT PRIMARY KEY, Origin_Station VARCHAR(100) NOT NULL, ...	0 row(s) affected
✓ 8	17:42:48	CREATE TABLE Schedules (Schedule_ID INT AUTO_INCREMENT PRIMARY KEY, Vehicle_ID INT NOT NULL, Rout...	0 row(s) affected
✓ 9	17:42:48	CREATE TABLE Bookings (Booking_ID INT AUTO_INCREMENT PRIMARY KEY, User_ID INT NOT NULL, Schedul...	0 row(s) affected
✓ 10	17:42:48	CREATE TABLE User_Phones (User_ID INT NOT NULL, Phone_Number VARCHAR(20) NOT NULL, PRIMARY KEY (...	0 row(s) affected
✓ 11	17:42:48	CREATE TABLE Luggage (Luggage_ID INT AUTO_INCREMENT PRIMARY KEY, Booking_ID INT NOT NULL, Tag_...	0 row(s) affected
✗ 12	17:42:48	CREATE TABLE Luggage (Luggage_ID INT AUTO_INCREMENT PRIMARY KEY, Booking_ID INT NOT NULL, Tag_...	Error Code: 1050. Table 'luggage' already exists

INSERT Statements

The following Data Manipulation (DML) queries inject realistic, structured dummy data into the database to validate all relational foreign key constraints.

1. Insert Users

INSERT INTO Users (First_Name, Last_Name, City, User_Role)
VALUES

('Ali', 'Hamza', 'Lahore', 'Passenger'),
('Ayesha', 'Tariq', 'Lahore', 'Passenger'),
('Fatima', 'Noor', 'Lahore', 'Passenger'),
('Ahmed', 'Bilal', 'Karachi', 'Passenger'),
('Zainab', 'Bibi', 'Islamabad', 'Passenger'),
('Muhammad', 'Mustafa', 'Rawalpindi', 'Passenger'),

('Sana', 'Khan', 'Multan', 'Passenger'),
 ('Hamza', 'Ali', 'Peshawar', 'Passenger'),
 ('Omer', 'Farooq', 'Faisalabad', 'Passenger'),
 ('Amna', 'Sheikh', 'Sialkot', 'Passenger'),
 ('Bilal', 'Ashraf', 'Gujranwala', 'Passenger'),
 ('Mariam', 'Javed', 'Quetta', 'Passenger'),
 ('Usman', 'Ghani', 'Lahore', 'Passenger'),
 ('Zain', 'Raza', 'Lahore', 'Passenger'),
 ('Khadija', 'Rehman', 'Karachi', 'Passenger'),
 ('Abdullah', 'Siddiqui', 'Islamabad', 'Passenger'),
 ('Hira', 'Mani', 'Lahore', 'Passenger'),
 ('Zeeshan', 'Haider', 'Multan', 'Passenger'),
 ('Nadia', 'Hussain', 'Faisalabad', 'Admin'),
 ('Asif', 'Ali', 'Sahiwal', 'Staff');

2. Insert Vehicles

INSERT INTO Vehicles (License_Plate, Vehicle_Type, Total_Capacity)
VALUES

('LHR-1001', 'AC Coaster', 25),
 ('LHR-1002', 'AC Coaster', 25),
 ('LHR-1003', 'AC Coaster', 25),
 ('LHR-1004', 'AC Coaster', 25),
 ('LHR-2005', 'Luxury Bus', 40),
 ('LHR-2006', 'Luxury Bus', 40),
 ('LHR-2007', 'Luxury Bus', 40),
 ('LHR-2008', 'Luxury Bus', 40),
 ('LHR-3040', 'Campus Shuttle', 15),
 ('LHR-3041', 'Campus Shuttle', 15),
 ('LHR-3042', 'Campus Shuttle', 15),
 ('LHR-3043', 'Campus Shuttle', 15),
 ('LHR-4001', 'Standard Bus', 50),
 ('LHR-4002', 'Standard Bus', 50),
 ('LHR-4003', 'Standard Bus', 50),
 ('LHR-4004', 'Standard Bus', 50),
 ('LHR-5001', 'Hi-Ace Van', 14),

```
('LHR-5002', 'Hi-Ace Van', 14),
('LHR-5003', 'Hi-Ace Van', 14),
('LHR-5004', 'Hi-Ace Van', 14);
```

3. Insert Routes

```
INSERT INTO Routes (Origin_Station, Destination_Station, Estimated_Duration_Hours)
VALUES
```

```
('Lahore Campus', 'Islamabad Campus', 4.50),
('Lahore Campus', 'Multan Terminal', 5.00),
('Lahore Campus', 'Faisalabad Terminal', 2.50),
('Lahore Campus', 'Gujranwala Stop', 1.50),
('Lahore Campus', 'Sialkot Terminal', 2.50),
('Lahore Campus', 'Rawalpindi Stop', 4.00),
('Lahore Campus', 'Peshawar Terminal', 6.00),
('Lahore Campus', 'Sargodha Stop', 3.00),
('Lahore Campus', 'Bahawalpur Terminal', 6.50),
('Lahore Campus', 'Sahiwal Stop', 2.50),
('Hostel Block A', 'Engineering Dept', 0.25),
('Hostel Block B', 'Main Library', 0.15),
('Main Gate', 'Business School', 0.10),
('Medical College', 'Sports Complex', 0.20),
('Girls Hostel', 'CS Department', 0.12),
('Staff Colony', 'Administration Block', 0.08),
('Main Library', 'Social Sciences Block', 0.05),
('Transport Pool', 'Auditoriums', 0.10),
('Main Gate', 'Girls Hostel', 0.18),
('Hostel Block A', 'Sports Complex', 0.22);
```

4. Insert Schedules

```
INSERT INTO Schedules (Vehicle_ID, Route_ID, Departure_Time, Ticket_Price,
Available_Seats)
```

```
VALUES
```

```
(1, 1, '2026-06-10 08:00:00', 2500.00, 25),
(5, 2, '2026-06-11 10:00:00', 1800.00, 40),
```

```
(2, 3, '2026-06-12 09:00:00', 1200.00, 25),
(17, 4, '2026-06-13 14:00:00', 800.00, 14),
(6, 5, '2026-06-14 15:00:00', 1500.00, 40),
(13, 6, '2026-06-15 07:00:00', 2000.00, 50),
(14, 7, '2026-06-16 06:00:00', 3000.00, 50),
(7, 8, '2026-06-17 11:00:00', 1000.00, 40),
(15, 9, '2026-06-18 21:00:00', 2800.00, 50),
(8, 10, '2026-06-19 13:00:00', 900.00, 40),
(9, 11, '2026-06-01 07:30:00', 50.00, 15),
(10, 12, '2026-06-01 07:45:00', 50.00, 15),
(18, 13, '2026-06-01 08:00:00', 30.00, 14),
(11, 14, '2026-06-01 17:00:00', 40.00, 15),
(12, 15, '2026-06-01 08:15:00', 35.00, 15),
(19, 16, '2026-06-01 08:30:00', 25.00, 14),
(20, 17, '2026-06-01 09:00:00', 20.00, 14),
(3, 18, '2026-06-01 12:00:00', 35.00, 25),
(4, 19, '2026-06-01 14:00:00', 45.00, 25),
(16, 20, '2026-06-01 16:30:00', 40.00, 50);
```

5. Insert Bookings

```
INSERT INTO Bookings (User_ID, Schedule_ID, Seats_Booked, Total_Amount,
Booking_Status)
```

```
VALUES
```

```
(1, 1, 2, 5000.00, 'Confirmed'),
(2, 2, 1, 1800.00, 'Confirmed'),
(1, 3, 3, 3600.00, 'Confirmed'),
(4, 4, 1, 800.00, 'Confirmed'),
(5, 5, 2, 3000.00, 'Confirmed'),
(6, 6, 1, 2000.00, 'Confirmed'),
(7, 7, 2, 6000.00, 'Confirmed'),
(8, 8, 4, 4000.00, 'Confirmed'),
(9, 9, 1, 2800.00, 'Confirmed'),
(10, 10, 2, 1800.00, 'Confirmed'),
(11, 11, 1, 50.00, 'Confirmed'),
(12, 12, 1, 50.00, 'Confirmed'),
```

```
(13, 13, 2, 60.00, 'Confirmed'),
(14, 14, 1, 40.00, 'Confirmed'),
(15, 15, 3, 105.00, 'Confirmed'),
(16, 16, 1, 25.00, 'Confirmed'),
(17, 17, 1, 20.00, 'Confirmed'),
(18, 18, 2, 70.00, 'Confirmed'),
(1, 19, 1, 45.00, 'Confirmed'),
(2, 20, 2, 80.00, 'Confirmed');
```

6. Insert User_phone

```
INSERT INTO User_Phones (User_ID, Phone_Number)
VALUES
(1, '0300-1112223'), (1, '0300-1111111'), (1, '042-9999999'),
(2, '0321-4445556'), (2, '0333-2222222'),
(3, '0333-7778889'), (3, '0300-3333333'),
(4, '0301-1234567'), (4, '0321-4444444'), (4, '042-4444444'),
(5, '0322-7654321'), (5, '0345-5555555'),
(6, '0345-9876543'), (6, '0311-6666666'),
(7, '0312-3456789'), (7, '0300-7777777'),
(8, '0300-9876543'), (8, '0333-8888888'),
(9, '0334-1122334'), (9, '0321-9999999'),
(10, '0321-5566778'), (10, '0312-0000000'),
(11, '0302-9988776'), (11, '0301-1234567'),
(12, '0313-4433221'), (12, '0334-7654321'),
(13, '0324-1111222'), (13, '0345-1122334'),
(14, '0305-5554443'), (14, '0322-5566778'),
(15, '0336-8889990'), (15, '0300-8899001'),
(16, '0307-6667778'), (16, '0331-4455667'),
(17, '0320-2223334'), (17, '0346-7788990'),
(18, '0346-4443332'), (18, '0305-2233445'),
(19, '0311-5556667'),
(20, '0300-0001112');
```

7. Insert Luggage

INSERT INTO Luggage (Booking_ID, Tag_Number, Weight_KG)
VALUES

(1, 1, 15.50), (1, 2, 8.00),
(2, 1, 12.00), (3, 1, 20.00),
(3, 2, 5.00), (3, 3, 9.25),
(4, 1, 10.50), (5, 1, 18.00),
(5, 2, 14.00), (6, 1, 11.50),
(7, 1, 22.00), (7, 2, 7.50),
(8, 1, 15.00), (8, 2, 15.00),
(8, 3, 12.00), (9, 1, 9.00),
(10, 1, 16.50), (13, 1, 14.00),
(15, 1, 25.00), (20, 1, 13.50);

admins_drivers x SQLAddition

Limit to 1000 rows

43 ('Asif', 'Ali', 'Sahiwal', 'Staff');
44
45
46
47 • INSERT INTO Vehicles (License_Plate, Vehicle_Type, Total_Capacity)
48
49 VALUES
50
51 ('LHR-1001', 'AC Coaster', 25),
52
53 ('LHR-1002', 'AC Coaster', 25),
54
55 ('LHR-1003', 'AC Coaster', 25),
56
57 ('LHR-1004', 'AC Coaster', 25).

Aut manual

Context Help

Output

Action Output

#	Time	Action	Message
5	17:42:47	CREATE TABLE Users (User_ID INT AUTO_INCREMENT PRIMARY KEY, First_Name VARCHAR(50) NOT NULL, ...	0 row(s) affected
6	17:42:47	CREATE TABLE Vehicles (Vehicle_ID INT AUTO_INCREMENT PRIMARY KEY, License_Plate VARCHAR(20) UNIQUE N...	0 row(s) affected
7	17:42:47	CREATE TABLE Routes (Route_ID INT AUTO_INCREMENT PRIMARY KEY, Origin_Station VARCHAR(100) NOT NULL...	0 row(s) affected
8	17:42:48	CREATE TABLE Schedules (Schedule_ID INT AUTO_INCREMENT PRIMARY KEY, Vehicle_ID INT NOT NULL, Ro...	0 row(s) affected
9	17:42:48	CREATE TABLE Bookings (Booking_ID INT AUTO_INCREMENT PRIMARY KEY, User_ID INT NOT NULL, Sched...	0 row(s) affected
10	17:42:48	CREATE TABLE User_Phones (User_ID INT NOT NULL, Phone_Number VARCHAR(20) NOT NULL, PRIMARY KEY...	0 row(s) affected
11	17:42:48	CREATE TABLE Luggage (Luggage_ID INT AUTO_INCREMENT PRIMARY KEY, Booking_ID INT NOT NULL, Ta...	0 row(s) affected
12	17:42:48	CREATE TABLE Luggage (Luggage_ID INT AUTO_INCREMENT PRIMARY KEY, Booking_ID INT NOT NULL, Ta...	Error Code: 1050. Table 'luggage' already exists
13	17:48:00	INSERT INTO Users (First_Name, Last_Name, City, User_Role) VALUES ('Ali', 'Hamza', 'Lahore', 'Passenger'), ('Ayesha', 'Tari...	20 row(s) affected Records: 20 Duplicates: 0 Warnings: 0
14	17:48:00	INSERT INTO Vehicles (License_Plate, Vehicle_Type, Total_Capacity) VALUES ('LHR-1001', 'AC Coaster', 25), ('LHR-1002', ...	20 row(s) affected Records: 20 Duplicates: 0 Warnings: 0
15	17:48:00	INSERT INTO Routes (Origin_Station, Destination_Station, Estimated_Duration_Hours) VALUES ('Lahore Campus', 'Islamabad ...	20 row(s) affected Records: 20 Duplicates: 0 Warnings: 0
16	17:48:00	INSERT INTO Schedules (Vehicle_ID, Route_ID, Departure_Time, Ticket_Price, Available_Seats) VALUES (1, 1, '2026-06-10 0...	20 row(s) affected Records: 20 Duplicates: 0 Warnings: 0
17	17:48:00	INSERT INTO Bookings (User_ID, Schedule_ID, Seats_Booked, Total_Amount, Booking_Status) VALUES (1, 1, 2, 5000.00, 'C...	20 row(s) affected Records: 20 Duplicates: 0 Warnings: 0
18	17:48:00	INSERT INTO User_Phones (User_ID, Phone_Number) VALUES (1, '0300-1112223'), (1, '0300-1111111'), (1, '042-9999999'), ...	40 row(s) affected Records: 40 Duplicates: 0 Warnings: 0
19	17:48:00	INSERT INTO Luggage (Booking_ID, Tag_Number, Weight_KG) VALUES (1, 1, 15.50), (1, 2, 8.00), (2, 1, 12.00), (3, 1, 20.0...	20 row(s) affected Records: 20 Duplicates: 0 Warnings: 0

9. Database Implementation and SQL Queries

Query 1: Passenger Search by City (WHERE & LIKE & AND operator)

Question Statement: Write an SQL query to display the User ID, first name, last name, city, and user role for all records in the system where the city is 'Lahore' and the role is 'Passenger'.

SQL

```
SELECT User_ID, First_Name, Last_Name, City, User_Role  
FROM Users  
WHERE City LIKE 'Lahore' AND User_Role = 'Passenger';
```

admins_drivers x

Limit to 1000 rows

```

1 • SELECT User_ID, First_Name, Last_Name, City, User_Role
2
3 FROM Users
4
5 WHERE City LIKE 'Lahore' AND User_Role = 'Passenger';

```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

	User_ID	First_Name	Last_Name	City	User_Role
▶	1	Ali	Hamza	Lahore	Passenger
	2	Ayesha	Tariq	Lahore	Passenger
	3	Fatima	Noor	Lahore	Passenger
	13	Usman	Ghani	Lahore	Passenger
	14	Zain	Raza	Lahore	Passenger
	17	Hira	Mani	Lahore	Passenger
*	NULL	NULL	NULL	NULL	NULL

Users 1 ▾

Query 2: Available Fleet Capacity Filter (WHERE & ORDER BY)

Question Statement: Write an SQL query to retrieve the Vehicle ID, license plate, type, and total capacity of all vehicles classified as a 'Luxury Bus', sorted by their capacity from highest to lowest.

SQL

```

SELECT Vehicle_ID, License_Plate, Vehicle_Type, Total_Capacity
FROM Vehicles
WHERE Vehicle_Type = 'Luxury Bus'
ORDER BY Total_Capacity DESC;

```


admins_drivers x

Limit to 1000 rows

```

1 • SELECT Vehicle_ID, License_Plate, Vehicle_Type, Total_Capacity
2
3 FROM Vehicles
4
5 WHERE Vehicle_Type = 'Luxury Bus'
6
7 ORDER BY Total_Capacity DESC;

```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap C

	Vehicle_ID	License_Plate	Vehicle_Type	Total_Capacity
▶	5	LHR-2005	Luxury Bus	40
	6	LHR-2006	Luxury Bus	40
	7	LHR-2007	Luxury Bus	40
	8	LHR-2008	Luxury Bus	40
*	NULL	NULL	NULL	NULL

Query 3: Passenger Booking List (JOIN)

Question Statement: Write an SQL query to generate a report showing the Booking ID, passenger's first name, last name, seats booked, and total amount paid by joining the bookings and users tables.

SQL

```

SELECT b.Booking_ID, u.First_Name, u.Last_Name, b.Seats_Booked, b.Total_Amount
FROM Bookings b
JOIN Users u ON b.User_ID = u.User_ID;

```

admins_drivers x

Limit to 1000 rows

```

1 • SELECT b.Booking_ID, u.First_Name, u.Last_Name, b.Seats_Booked, b.Total_Amount
2
3 FROM Bookings b
4
5 JOIN Users u ON b.User_ID = u.User_ID;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: ☐

	Booking_ID	First_Name	Last_Name	Seats_Booked	Total_Amount
▶	1	Ali	Hamza	2	5000.00
	2	Ayesha	Tariq	1	1800.00
	3	Ali	Hamza	3	3600.00
	4	Ahmed	Bilal	1	800.00
	5	Zainab	Bibi	2	3000.00
	6	Muhammad	Mustafa	1	2000.00
	7	Sana	Khan	2	6000.00
	8	Hamza	Ali	4	4000.00
	9	Omer	Farooq	1	2800.00
	10	Amna	Sheikh	2	1800.00
	11	Bilal	Ashraf	1	50.00
	12	Mariam	Javed	1	50.00
	13	Usman	Ghani	2	60.00
	14	Zain	Raza	1	40.00
	15	Khadija	Rehman	3	105.00
	16	Abdullah	Siddiqui	1	25.00
	17	Hira	Mani	1	20.00
	18	Zeeshan	Haider	2	70.00
	19	Ali	Hamza	1	45.00
	20	Ayesha	Tariq	2	80.00

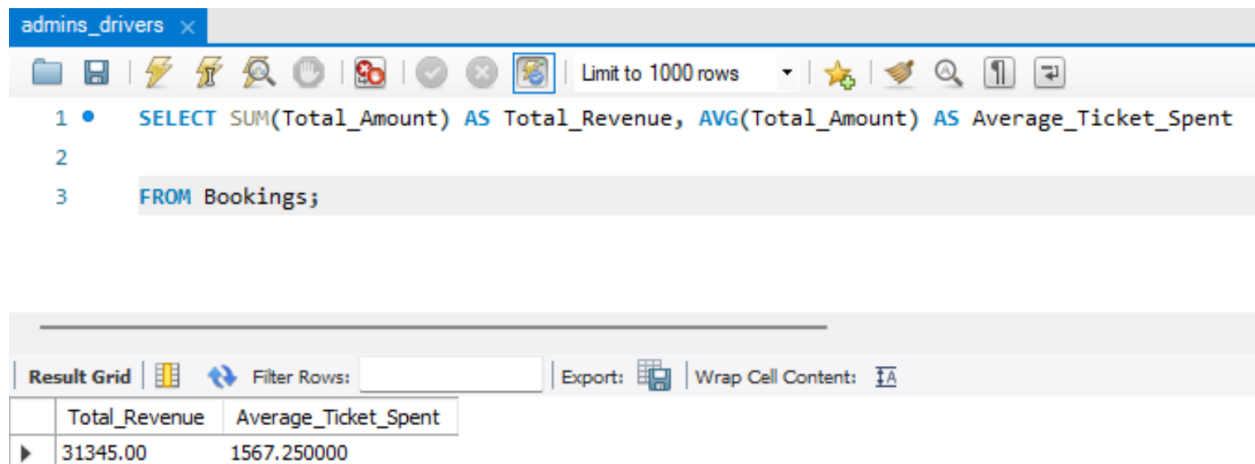
Query 4: Total Revenue and Average Ticket Price (Aggregate Functions: SUM and AVG)

Question Statement: Write an SQL query using aggregate functions to calculate the total financial revenue collected from all bookings combined alongside the average amount spent per transaction.

SQL

```
SELECT SUM(Total_Amount) AS Total_Revenue, AVG(Total_Amount) AS
Average_Ticket_Spent
```

FROM Bookings;



The screenshot shows a database query editor window titled 'admins_drivers'. The query editor contains the following SQL query:

```
1 • SELECT SUM(Total_Amount) AS Total_Revenue, AVG(Total_Amount) AS Average_Ticket_Spent
2
3 FROM Bookings;
```

Below the query editor, the 'Result Grid' is displayed, showing the results of the query. The grid has two columns: 'Total_Revenue' and 'Average_Ticket_Spent'. The results are as follows:

Total_Revenue	Average_Ticket_Spent
31345.00	1567.250000

Query 5: Finding Specific Vehicle Types (Fixed to capture exact 'AC Coaster' string matches)

Question Statement: Write an SQL query using the IN operator to display the Vehicle ID, license plate, and vehicle type for all vehicles that are either a 'Luxury Bus' or an 'AC Coaster'.

SQL

```
SELECT Vehicle_ID, License_Plate, Vehicle_Type
FROM Vehicles
WHERE Vehicle_Type IN ('Luxury Bus', 'AC Coaster');
```

The screenshot shows a database query editor window titled 'admins_drivers'. The query is as follows:

```

1 • SELECT Vehicle_ID, License_Plate, Vehicle_Type
2
3 FROM Vehicles
4
5 WHERE Vehicle_Type IN ('Luxury Bus', 'AC Coaster');

```

Below the query editor, the 'Result Grid' is displayed, showing the results of the query. The grid has three columns: Vehicle_ID, License_Plate, and Vehicle_Type. The results are as follows:

Vehicle_ID	License_Plate	Vehicle_Type
1	LHR-1001	AC Coaster
2	LHR-1002	AC Coaster
3	LHR-1003	AC Coaster
4	LHR-1004	AC Coaster
5	LHR-2005	Luxury Bus
6	LHR-2006	Luxury Bus
7	LHR-2007	Luxury Bus
8	LHR-2008	Luxury Bus
NULL	NULL	NULL

Query 6: Counting Schedules per Route (GROUP BY and HAVING)

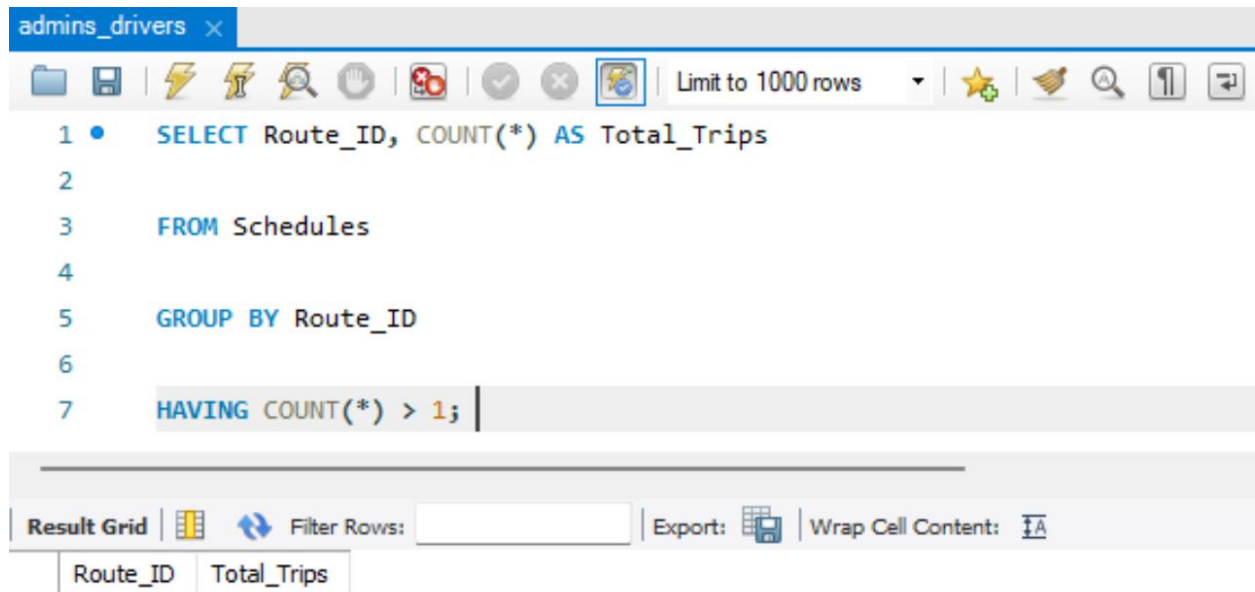
Question Statement: Write an SQL query to display each Route ID and the total number of trips scheduled for it, filtering the grouped results to only show routes that appear more than once in the schedule history.

SQL

```

SELECT Route_ID, COUNT(*) AS Total_Trips
FROM Schedules
GROUP BY Route_ID
HAVING COUNT(*) > 1;

```



The screenshot shows a SQL query editor window titled 'admins_drivers'. The query is as follows:

```
1 • SELECT Route_ID, COUNT(*) AS Total_Trips
2
3 FROM Schedules
4
5 GROUP BY Route_ID
6
7 HAVING COUNT(*) > 1;
```

Below the query editor, there is a toolbar with options like 'Result Grid', 'Filter Rows', 'Export', and 'Wrap Cell Content'. A small table preview is visible below the toolbar:

Route_ID	Total_Trips
----------	-------------

Query 7: High-Value Transactions (Subquery)

Question Statement: Write an SQL query utilizing a subquery to extract the Booking ID, User ID, and total amount for all transactions where the payment value is strictly higher than the overall average booking amount.

SQL

```
SELECT Booking_ID, User_ID, Total_Amount
FROM Bookings
WHERE Total_Amount > (SELECT AVG(Total_Amount) FROM Bookings);
```

```
admins_drivers x
1 • SELECT Booking_ID, User_ID, Total_Amount
2
3 FROM Bookings
4
5 WHERE Total_Amount > (SELECT AVG(Total_Amount) FROM Bookings);
```

Result Grid			
Filter Rows:			
Edit:			
Export/Import:			
Wrap Cell Content:			
	Booking_ID	User_ID	Total_Amount
▶	1	1	5000.00
	2	2	1800.00
	3	1	3600.00
	5	5	3000.00
	6	6	2000.00
	7	7	6000.00
	8	8	4000.00
	9	9	2800.00
	10	10	1800.00
*	NULL	NULL	NULL

Query 8: The ALTER Command

Question Statement: Write an SQL query to add a new column named Passenger_Email with a data type of VARCHAR(100) to the existing Users table.

SQL

ALTER TABLE Users

ADD Passenger_Gender VARCHAR(8);

```
admins_drivers x
1 • ALTER TABLE Users
2
3 ADD Passenger_Gender VARCHAR(8);
4
```

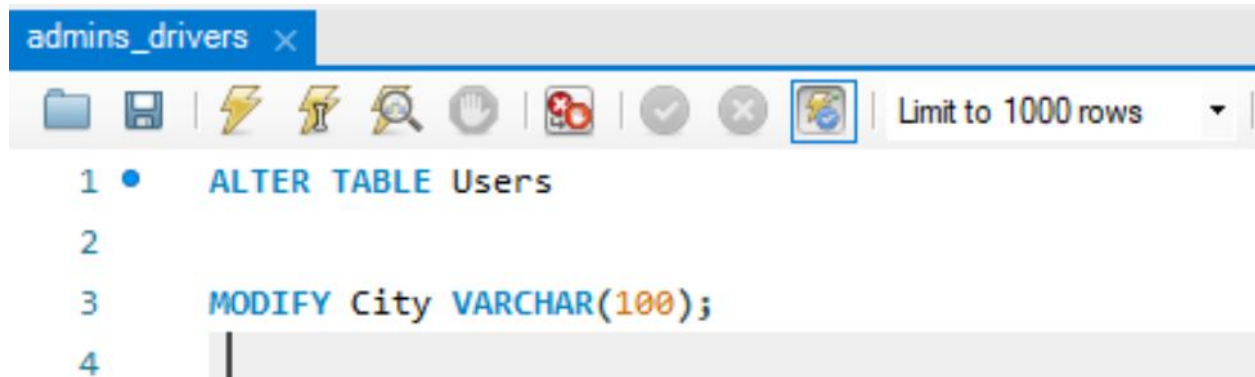
Query 9: The MODIFY Command

Question Statement: Write an SQL query to modify the existing City column in the Users table to expand its character length to VARCHAR(100).

SQL

ALTER TABLE Users

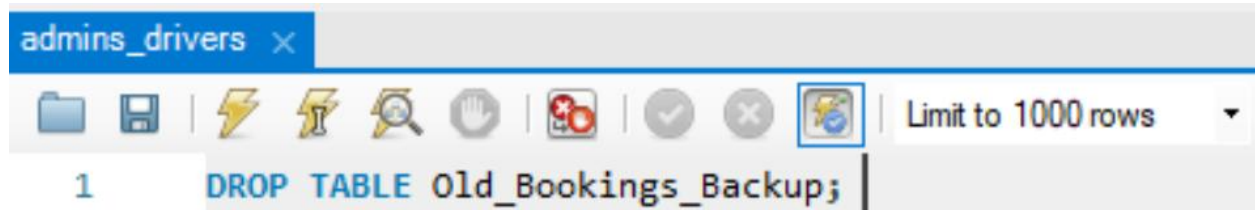
MODIFY City VARCHAR(100);



Query 10: The DROP Command

Question Statement: Write an SQL query to permanently delete a temporary backup table named Old_Bookings_Backup from the database structure.

SQL



Query 11: The UPDATE Command

Question Statement: Write an SQL query to modify the passenger records so that the user with a User_ID of 2 has their recorded City changed to 'Islamabad'.

SQL

UPDATE Users

SET City = 'Islamabad'

WHERE User_ID = 2;

```

1  UPDATE Users
2  SET City = 'Islamabad'
3  WHERE User_ID = 2;
4  select * from users
  
```

	User_ID	First_Name	Last_Name	City	User_Role	Passenger_Gender
▶	1	Ali	Hamza	Lahore	Passenger	NULL
	2	Ayesha	Tariq	Islamabad	Passenger	NULL

Query 12: The DELETE Command

Question Statement: Write an SQL query to remove a specific record from the Bookings table where the Booking_ID is equal to 5.

SQL

DELETE FROM Bookings

WHERE Booking_ID = 5;

admins_drivers x

Limit to 1000 rows

```

1 • DELETE FROM Bookings
2
3   WHERE Booking_ID = 5;
4 • select * from bookings

```

Result Grid

	Booking_ID	User_ID	Schedule_ID	Seats_Booked	Total_Amount	Booking_Status	Booking_Date
▶	1	1	1	2	5000.00	Confirmed	2026-06-01 17:48:00
	2	2	2	1	1800.00	Confirmed	2026-06-01 17:48:00
	3	1	3	3	3600.00	Confirmed	2026-06-01 17:48:00
	4	4	4	1	800.00	Confirmed	2026-06-01 17:48:00
	6	6	6	1	2000.00	Confirmed	2026-06-01 17:48:00

Query 13: The TRUNCATE Command

Question Statement: Write an SQL query to remove all records from a temporary testing log table named Test_Logs instantly, ensuring the table structure remains completely empty and available for future use.

SQL

TRUNCATE TABLE Test_Logs;

admins_drivers x

Limit to 1000 rows

```

1 • TRUNCATE TABLE Test_Logs;

```

Query 14: The EXISTS Operator

Question Statement: Write an SQL query to list the User ID, first name, and last name of all users in the system who have placed at least one booking, utilizing the EXISTS operator.

SQL

SELECT User_ID, First_Name, Last_Name

FROM Users u

WHERE EXISTS (

SELECT 1

FROM Bookings b

WHERE b.User_ID = u.User_ID

);

The screenshot shows a SQL query editor window titled 'admins_drivers'. The query is as follows:

```

1  SELECT User_ID, First_Name, Last_Name
2
3  FROM Users u
4
5  WHERE EXISTS (
6
7      SELECT 1
8
9      FROM Bookings b
10

```

Below the query editor is the 'Result Grid' showing 18 rows of data. The columns are User_ID, First_Name, and Last_Name.

User_ID	First_Name	Last_Name
1	Ali	Hamza
2	Ayesha	Tariq
4	Ahmed	Bilal
6	Muhammad	Mustafa
7	Sana	Khan
8	Hamza	Ali
9	Omer	Farooq
10	Amna	Sheikh
11	Bilal	Ashraf
12	Mariam	Javed
13	Usman	Ghani
14	Zain	Raza
15	Khadija	Rehman
16	Abdullah	Siddiqui
17	Hira	Mani
18	Zeeshan	Haider

Query 15: The NOT EXISTS Operator

Question Statement: Write an SQL query to find the names of all registered accounts whose role is 'Passenger' but who have never booked a single ticket, using the NOT EXISTS operator.

SQL

SELECT User_ID, First_Name, Last_Name

FROM Users u

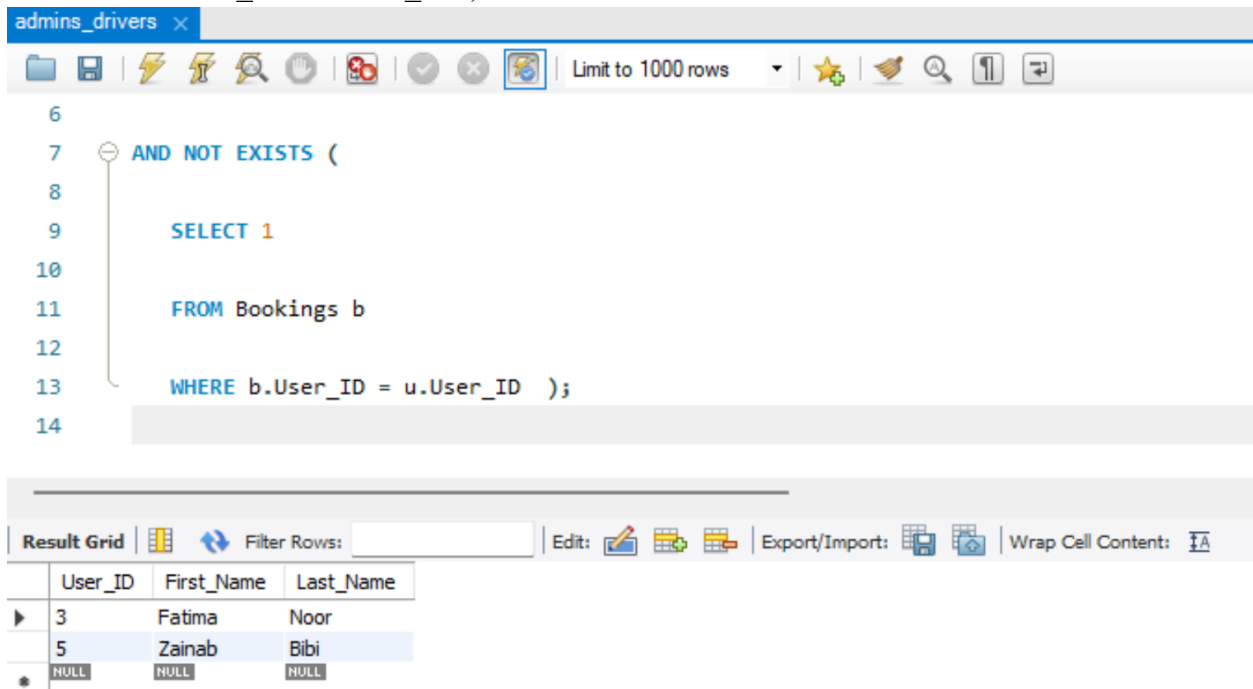
WHERE u.User_Role = 'Passenger'

AND NOT EXISTS (

```

SELECT 1
FROM Bookings b
WHERE b.User_ID = u.User_ID );

```



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The query editor displays the following SQL code:

```

6
7  AND NOT EXISTS (
8
9      SELECT 1
10
11     FROM Bookings b
12
13     WHERE b.User_ID = u.User_ID );
14

```

Below the query editor is the 'Result Grid' tab. It shows a table with three columns: User_ID, First_Name, and Last_Name. The data is as follows:

User_ID	First_Name	Last_Name
3	Fatima	Noor
5	Zainab	Bibi
NULL	NULL	NULL

The bottom toolbar includes options for 'Filter Rows', 'Edit', 'Export/Import', and 'Wrap Cell Content'.

Query 16: Complex Nested Subquery

Question Statement: Write an SQL query utilizing nested subqueries across multiple tables to find the first name and last name of all passengers who have booked tickets on a schedule that is actively assigned to a 'Luxury Bus'.

SQL

```

SELECT First_Name, Last_Name
FROM Users
WHERE User_ID IN (
    SELECT User_ID
    FROM Bookings
    WHERE Schedule_ID IN (
        SELECT Schedule_ID
        FROM Schedules

```

```
WHERE Vehicle_ID IN (  
    SELECT Vehicle_ID  
    FROM Vehicles  
    WHERE Vehicle_Type = 'Luxury Bus'  
)  
)  
);
```

admins_drivers x

Limit to 1000 rows

```
1 • SELECT First_Name, Last_Name  
2  
3 FROM Users  
4  
5 WHERE User_ID IN (  
6  
7     SELECT User_ID  
8  
9     FROM Bookings  
10
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	First_Name	Last_Name
▶	Ayesha	Tariq
	Hamza	Ali
	Amna	Sheikh